Experiment 1

Utsav Golani
T12 32 AI&DS
ASD&D Lab

LO2:Use git effectively for version control sourcecode managements and collaborative managements

Introduction to Git and GitHub

Git is a distributed version control system used to track code changes, while GitHub is a cloud platform for hosting Git repositories. Git helps developers collaborate, manage versions, and maintain project history efficiently.

1. Navigating and Branching Setup

Theory: Branching allows developers to work on parallel versions of a project without affecting the primary codebase (`main`).

`cd turf`: A standard terminal command (Change Directory) to enter the local repository folder.

`git branch`: Lists all local branches.

Output: `* main`, `new`, `new1`. The asterisk and green text indicate that `main` is currently checked out (active).

2. Handling Rebase Blockers and Commits

Theory: Git protects your work. It will prevent operations like a rebase if you have unstaged or uncommitted changes that might get overwritten or cause complex conflicts.

`git branch -d new`: Deletes the branch named `new`.

Output: Confirms deletion of the branch at commit `c3474c6`.

`git rebase main`: Attempts to reapply current branch commits onto `main`.

Output: `error: cannot rebase`. Git blocks this because there are uncommitted changes in the index.

`git commit -m "x=2"`: Saves the uncommitted changes to the local history.

Output: `[main f90d228] x=2`. Creates a new commit with the hash `f90d228`.

`git rebase main`: Attempted again after committing.

Output: `Current branch main is up to date.` Since the active branch is already `main`, there is nothing to rebase.

3. Extensive Branch Management

Theory: Switching branches safely updates the working directory to reflect that specific branch's history. Creating a branch (`-c`) and switching to it simultaneously speeds up workflows.

`git branch turf11`: Creates a new branch named `turf11`.

`git switch turf11`: Moves the HEAD pointer to make `turf11` active.

`git switch -c turf1`: Creates a new branch named `turf1` and immediately switches to it.

`git pull origin main`: Fetches and merges the latest updates from the remote GitHub repository.

Output: `Already up to date`, meaning the local `main` matches the remote `main`.

`git stash` / `git stash list`: Attempts to save temporary work, but outputs `No local changes to save` because the working directory is clean.

`git merge turf11` / `git merge turf1` / `git merge new1`: Integrates the history of these branches into the current branch.

Output: `Already up to date` for all, indicating no new distinct commits were present on those branches.

`git branch -d new1`: Cleans up by deleting the merged `new1` branch.

4. Code Inspection and Authorship

Theory: When debugging or tracking changes in collaborative environments, it is critical to know exactly who introduced a specific line of code and when.

`git blame index.html` & `git blame style.css`: Analyzes the specified files line-by-line.

Output: A detailed pager view (using `less`) displaying the commit hash (`e2f619c7`), author name (`Utsav1x`), timestamp (`2025-07-24 10:25:05`), and the exact line of HTML/CSS code corresponding to that commit.

5. History, Debugging, and Tagging

Theory: Git retains a meticulous record of every action. `reflog` is the ultimate safety net for recovering lost states, while tags create permanent snapshots for software releases.

`git clean -fd`: Forcibly removes untracked files and directories to ensure a clean working state.

`git bisect start / good / bad / reset`: A binary search algorithm used to find the exact commit that introduced a bug. The commands here initialize it, mark good/bad states, and then abort/reset the process.

`git reflog`: Displays a local history of where the `HEAD` pointer has moved.

Output: Shows every checkout, merge, and commit (e.g., `moving from main to new`, `commit: x=2`).

`git tag v1.0.0` (up to `v1.1.0`): Creates semantic versioning tags pointing to the current commit.

Output: Running `git tag` alone lists all locally created tags.

`git log`: Displays the standard chronological commit history.

Output: Shows the commit hashes, branch pointers, tags, author details, dates, and commit messages (e.g., "added new python file").

6. Remote Synchronization

Theory: Local changes (commits and tags) do not automatically sync to GitHub. They must be explicitly pushed to the remote origin.

`git push origin --tags`: Specifically uploads local tags to the remote repository.

Output: Success messages showing objects compressed and written to GitHub, creating `[new tag]` references remotely.

`git status`: Checks the state of the repository.

Output: `Your branch is ahead of 'origin/main' by 2 commits.` This warns that local commits exist that haven't been uploaded yet.

`git push`: Standard push to upload the outstanding local commits.

Output: Synchronizes `c3474c6..f90d228` from local `main` to remote `main`. A subsequent `git status` confirms the tree is perfectly clean and up to date.

7. Interactive Rewriting and Rollbacks

Theory: Sometimes history needs to be modified before pushing (rebasing), or a mistake needs to be undone safely without destroying the chronological history (reverting).

`git rebase -i HEAD~3`: Opens an interactive rebase for the last 3 commits.

Output: `Successfully rebased and updated refs/heads/main.` Indicates the interactive session in Vim was completed successfully.

`git restore .`: Reverts any current, uncommitted modifications in the working directory back to the last commit state.

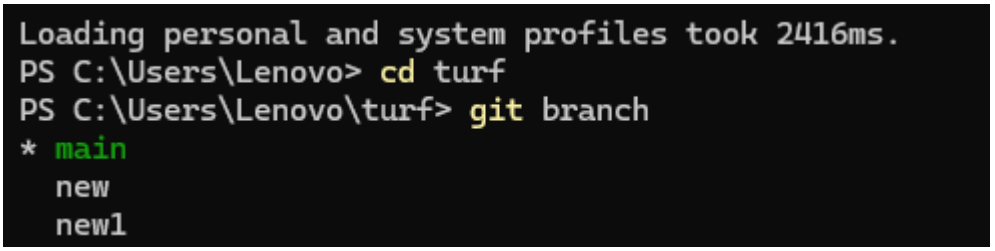
`git revert`: Executed without a target, so Git prints the usage instructions/help menu.

`git revert HEAD`: Creates a new commit that applies the exact inverse of the last commit (HEAD).

Output: `[main 4d7cff2] Revert "x=2"`. It successfully generated a new commit that removed the changes introduced previously, logging `1 file changed, 3 deletions(-)`.

Conclusion

Git provides a powerful set of commands for authentication, repository management, version control, branching, collaboration, and history tracking. Understanding these commands enables developers to work efficiently in individual and team environments.



```
PS C:\Users\Lenovo\turf> git branch -d new
Deleted branch new (was c3474c6).
PS C:\Users\Lenovo\turf> git rebase main
error: cannot rebase: Your index contains uncommitted changes.
error: Please commit or stash them.
PS C:\Users\Lenovo\turf> git commit -m "x=2"
[main f90d228] x=2
PS C:\Users\Lenovo\turf> git rebase main
Current branch main is up to date.
PS C:\Users\Lenovo\turf> git diff
PS C:\Users\Lenovo\turf> git diff main .. new1
fatal: new1: no such path in the working tree.
Use 'git <command> -- <path>...' to specify paths that do not exist locally.
```

```
PS C:\Users\Lenovo\turf> git clean -fd
PS C:\Users\Lenovo\turf> git bisect start
status: waiting for both 'good' and 'bad' commits
PS C:\Users\Lenovo\turf> git bisect good
status: waiting for 'bad' commit, 1 'good' commit known
PS C:\Users\Lenovo\turf> git bisect bad
f90d228255e53a8a16db4a6edd4e18399cbcaba4 was both 'good' and 'bad'
PS C:\Users\Lenovo\turf> git bisect reset
Already on 'main'
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)
PS C:\Users\Lenovo\turf> git reflog
f90d228 (HEAD -> main) HEAD@{0}: checkout: moving from main to main
f90d228 (HEAD -> main) HEAD@{1}: commit (merge): x=2
c3474c6 (origin/main, origin/HEAD) HEAD@{2}: checkout: moving from main to main
c3474c6 (origin/main, origin/HEAD) HEAD@{3}: checkout: moving from new1 to main
dfec0d6 (new1) HEAD@{4}: commit: x=2
560effc HEAD@{5}: reset: moving to HEAD~1
c3474c6 (origin/main, origin/HEAD) HEAD@{6}: reset: moving to HEAD
c3474c6 (origin/main, origin/HEAD) HEAD@{7}: checkout: moving from new to new1
c3474c6 (origin/main, origin/HEAD) HEAD@{8}: checkout: moving from main to new
c3474c6 (origin/main, origin/HEAD) HEAD@{9}: commit: added new python file
560effc HEAD@{10}: clone: from https://github.com/Utsav1x/turf.git
PS C:\Users\Lenovo\turf> git tag
PS C:\Users\Lenovo\turf> git tag v1.0.0
PS C:\Users\Lenovo\turf> git tag
v1.0.0
PS C:\Users\Lenovo\turf> git tag v1.0.1
PS C:\Users\Lenovo\turf> git tag v1.1.0
PS C:\Users\Lenovo\turf> git tag v1.0.2
PS C:\Users\Lenovo\turf> git tag
v1.0.0
v1.0.1
v1.0.2
v1.1.0
```

```
PS C:\Users\Lenovo\turf> git push origin --tags
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 385 bytes | 385.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/Utsav1x/turf.git
 * [new tag]         v1.0.0 -> v1.0.0
 * [new tag]         v1.0.1 -> v1.0.1
 * [new tag]         v1.0.2 -> v1.0.2
 * [new tag]         v1.1.0 -> v1.1.0
PS C:\Users\Lenovo\turf> git status
On branch main
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
PS C:\Users\Lenovo\turf> git push
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Utsav1x/turf.git
   c3474c6..f90d228  main -> main
PS C:\Users\Lenovo\turf> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

```
PS C:\Users\Lenovo\turf> git log
commit f90d228255e53a8a16db4a6edd4e18399cbcaba4 (HEAD -> main, tag: v1.1.0, tag: v1.0.2, tag: v1.0.1, tag: v1.0.0, origin/main, origin/HEAD)
Merge: c3474c6 dfec0d6
Author: Utsav <your.email@example.com>
Date:   Tue Jul 14 10:37:41 2026 +0530

    x=2

commit dfec0d68b0a802d3e5f773ad5e2626a8e96a9b20 (new1)
Author: Utsav <your.email@example.com>
Date:   Tue Jul 14 10:34:30 2026 +0530

    x=2

commit c3474c6faff5a044961a28bcabe40e04e6d14eda
Author: Utsav <your.email@example.com>
Date:   Tue Jul 14 10:24:18 2026 +0530

    added new python file

commit 560effc6eabd4c000e8bcb6acf061e73af4213a0
Author: Utsav1x <utsav.golani@gmail.com>
Date:   Thu Jul 24 13:43:42 2025 +0530

    Add files via upload

commit a13d00241ae284bd6c634436142626f3bfb9c547
Author: Utsav1x <utsav.golani@gmail.com>
Date:   Thu Jul 24 13:40:37 2025 +0530
```

```
PS C:\Users\Lenovo\turf> git branch
* main
  new1
PS C:\Users\Lenovo\turf> git branch turf11
PS C:\Users\Lenovo\turf> git branch
* main
  new1
  turf11
PS C:\Users\Lenovo\turf> git switch turf11
Switched to branch 'turf11'
PS C:\Users\Lenovo\turf> git branch
  main
  new1
* turf11
PS C:\Users\Lenovo\turf> git switch -c turf1
Switched to a new branch 'turf1'
PS C:\Users\Lenovo\turf> git pull origin main
From https://github.com/Utsav1x/turf
 * branch            main       -> FETCH_HEAD
Already up to date.
PS C:\Users\Lenovo\turf> git stash
No local changes to save
PS C:\Users\Lenovo\turf> git stash list
PS C:\Users\Lenovo\turf> git swich main
git: 'swich' is not a git command. See 'git --help'.

The most similar command is
        switch
PS C:\Users\Lenovo\turf> git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\Lenovo\turf> git merge turf11
Already up to date.
PS C:\Users\Lenovo\turf> git merge turf1
Already up to date.
```

```
PS C:\Users\Lenovo\turf> git merge main
Already up to date.
PS C:\Users\Lenovo\turf> git merge new
merge: new - not something we can merge
PS C:\Users\Lenovo\turf> git branch -d new
error: branch 'new' not found
PS C:\Users\Lenovo\turf> git branch
* main
  new1
  turf1
  turf11
PS C:\Users\Lenovo\turf> git merge new1
Already up to date.
PS C:\Users\Lenovo\turf> git branch -d new1
Deleted branch new1 (was dfec0d6).
PS C:\Users\Lenovo\turf> git rebase main
Current branch main is up to date.
PS C:\Users\Lenovo\turf> git diff
PS C:\Users\Lenovo\turf> git diff turf11 turf1
```

```
PS C:\Users\Lenovo\turf> git blame style.css
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530   1) * {
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530   2)     margin: 0;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530   3)     padding: 0;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530   4)     font-family: 'Mulish', sans-serif;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530   5)   }
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530   6)
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530   7)   /* navbar section */
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530   8)   .navbar {
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530   9)     box-shadow: 0 15px 40px -20px rgb(40 44 63 / 15%);
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  10)     position: fixed;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  11)     top: 0;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  12)     left: 0;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  13)     right: 0;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  14)     height: 80px;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  15)     background: #fff;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  16)     z-index: 1000;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  17)     padding: 0 20px;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  18)   }
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  19)   .navbar .nav {
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  20)     max-width: 1200px;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  21)     min-width: 1200px;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  22)     position: relative;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  23)     margin: 0 auto;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  24)     height: 80px;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  25)     background: #fff;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  26)     display: flex;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  27)     justify-content: space-between;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  28)   }
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  29)   .navbar .left {
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  30)     display: flex;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  31)     align-items: center;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  32)   }
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  33)   .navbar .left .logo {
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  34)     display: block;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  35)     height: 49px;
e2f619c7 (Utsav1x 2025-07-24 10:25:05 +0530  36)     transition: transform 0.3s cubic-bezier(0.215, 0.61, 0.355, 1);
```

```
PS C:\Users\Lenovo\turf> git clean -fd
PS C:\Users\Lenovo\turf> git rebase -i HEAD~3
Successfully rebased and updated refs/heads/main.
PS C:\Users\Lenovo\turf> git restore .
PS C:\Users\Lenovo\turf>   git revert
usage: git revert [--[no-]edit] [-n] [-m <parent-number>] [-s] [-S[<keyid>]] <commit>...
   or: git revert (--continue | --skip | --abort | --quit)

    --quit                end revert or cherry-pick sequence
    --continue            resume revert or cherry-pick sequence
    --abort               cancel revert or cherry-pick sequence
    --skip                skip current commit and continue
    --[no-]cleanup <mode> how to strip spaces and #comments from message
    -n, --no-commit       don't automatically commit
    --commit              opposite of --no-commit
    -e, --[no-]edit       edit the commit message
    -s, --[no-]signoff    add a Signed-off-by trailer
    -m, --[no-]mainline <parent-number>
                          select mainline parent
    --[no-]rerere-autoupdate
                          update the index with reused conflict resolution if possible
    --[no-]strategy <strategy>
                          merge strategy
    -X, --[no-]strategy-option <option>
                          option for merge strategy
    -S, --[no-]gpg-sign[=<key-id>]
                          GPG sign commit
    --[no-]reference      use the 'reference' format to refer to commits

PS C:\Users\Lenovo\turf> git revert HEAD
>>
[main 4d7cff2] Revert "x=2"
 1 file changed, 3 deletions(-)
```